

Fordítóprogramok készítése

BMEVIIIIML10 – Önállólaboratórium 1
Beszámoló

Bodor András
Készítette

Dr. Simon Balázs
Konzulens

○ Célok

- C és C++ projektek AST-jében való keresés
- Clang-Tidy, csak scriptelhető
- AWK, csak strukturált AST bemeneten
- Ehhez kell egy nyelv
 - Megfelelően rugalmas egy kereső DSL-hez
 - Könnyen megérthető
 - Nem (csak) fordított, JIT/Interpreter előnyösebb

LISP

- Nagyon általános/flexibilis szintaxis
- Sokaknak idegen
- Kicsit közelebbi kellene a “megszokott” nyelvekhez

```
1 + f(2 * 3)
```

```
(+ 1 (f (* 2 3)))
```

```
if (a > 2) {  
    print("a")  
}  
else {  
    print("b")  
}
```

```
(if (> a 2)  
    (print "a")  
    (print "b"))
```

○ C4 nyelv

- Flexibilis szintaktika
 - LISP+Haskell+Tcl+(kicsi)Ruby
- Minden függvény
 - Nincsenek* beépített nyelvi elemek
- Ez nem feltűnő
 - “if” megszokottan viselkedik, hiába függvény

```
if (a > 2) {  
    print("a")  
}  
else {  
    print("b")  
}
```

○ C4 nyelv

- Flexibilis szintaktika
 - LISP+Haskell+Tcl+(kicsi)Ruby
- Minden függvény
 - Nincsenek* beépített nyelvi elemek
- Ez nem feltűnő
 - “if” megszokottan viselkedik, hiába függvény

```
if a > 2 {  
    print "a"  
}  
else {  
    print "b"  
}
```

○ C4 nyelv

- Flexibilis szintaktika
 - LISP+Haskell+Tcl+(kicsi)Ruby
- Minden függvény
 - Nincsenek* beépített nyelvi elemek
- Ez nem feltűnő
 - “if” megszokottan viselkedik, hiába függvény

print/1

```
if a > 2 {  
    print "a"  
}  
else {  
    print "b"  
}
```

○ C4 nyelv

- Flexibilis szintaktika
 - LISP+Haskell+Tcl+(kicsi)Ruby
- Minden függvény
 - Nincsenek* beépített nyelvi elemek
- Ez nem feltűnő
 - “if” megszokottan viselkedik, hiába függvény

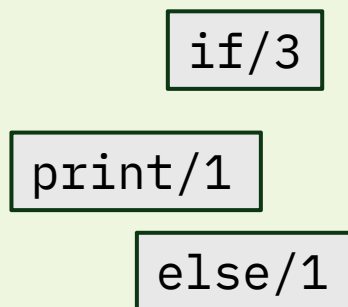
if/3

print/1

```
if a > 2 {  
    print "a"  
}  
else {  
    print "b"  
}
```

○ C4 nyelv

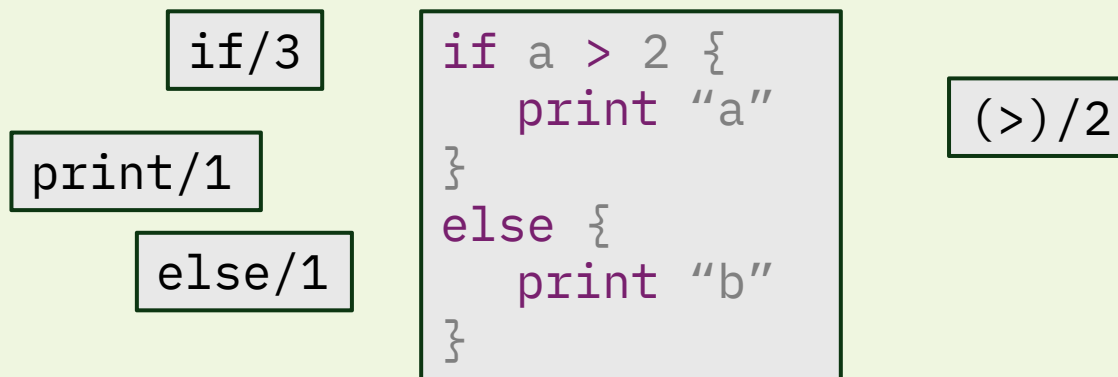
- Flexibilis szintaktika
 - LISP+Haskell+Tcl+(kicsi)Ruby
- Minden függvény
 - Nincsenek* beépített nyelvi elemek
- Ez nem feltűnő
 - “if” megszokottan viselkedik, hiába függvény



```
if a > 2 {  
    print "a"  
}  
else {  
    print "b"  
}
```

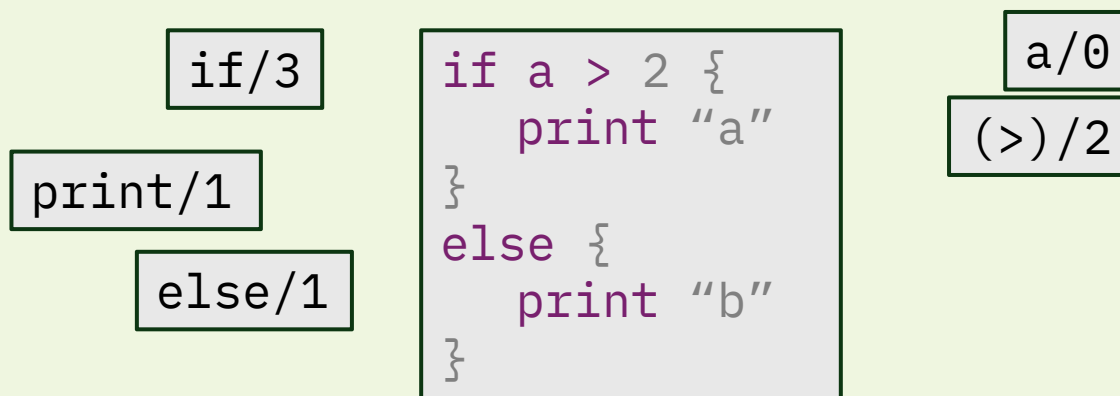

○ C4 nyelv

- Flexibilis szintaktika
 - LISP+Haskell+Tcl+(kicsi)Ruby
- Minden függvény
 - Nincsenek* beépített nyelvi elemek
- Ez nem feltűnő
 - “if” megszokottan viselkedik, hiába függvény



○ C4 nyelv

- Flexibilis szintaktika
 - LISP+Haskell+Tcl+(kicsi)Ruby
- Minden függvény
 - Nincsenek* beépített nyelvi elemek
- Ez nem feltűnő
 - “if” megszokottan viselkedik, hiába függvény



○ C4 nyelv II

- Egy kulcsszó: **let**
- Függvények definiálására
 - Változó is függvény
 - Operátor is függvény

```
let fun/2 { |a b| ... }  
# fun 1 2  
let (>_<)/2 left 2 { |a b| ... }  
# 1 >_< 2  
let (^-^)/1 { |x| ... }  
# ^-^123  
let x 123  
# x
```

○ Fordítás

- LLVM backend
 - Sok architektúrát támogat (x86(-64), ARM(64), RISC-V, SPARC, z/Architecture, ...)
- Optimalizálás
 - “Ingyen” kapjuk LLVM mellé
 - Várhatóan jobb, mint amit mi tudnánk írni
- Egyelőre compiler
 - JIT támogatás folyamatban

◉ LLVM Internal Representation

- LLVM támogatáshoz
- Alacsony szintű nyelv
 - Regiszterek
 - Manuális stack allokálás
 - SSA forma

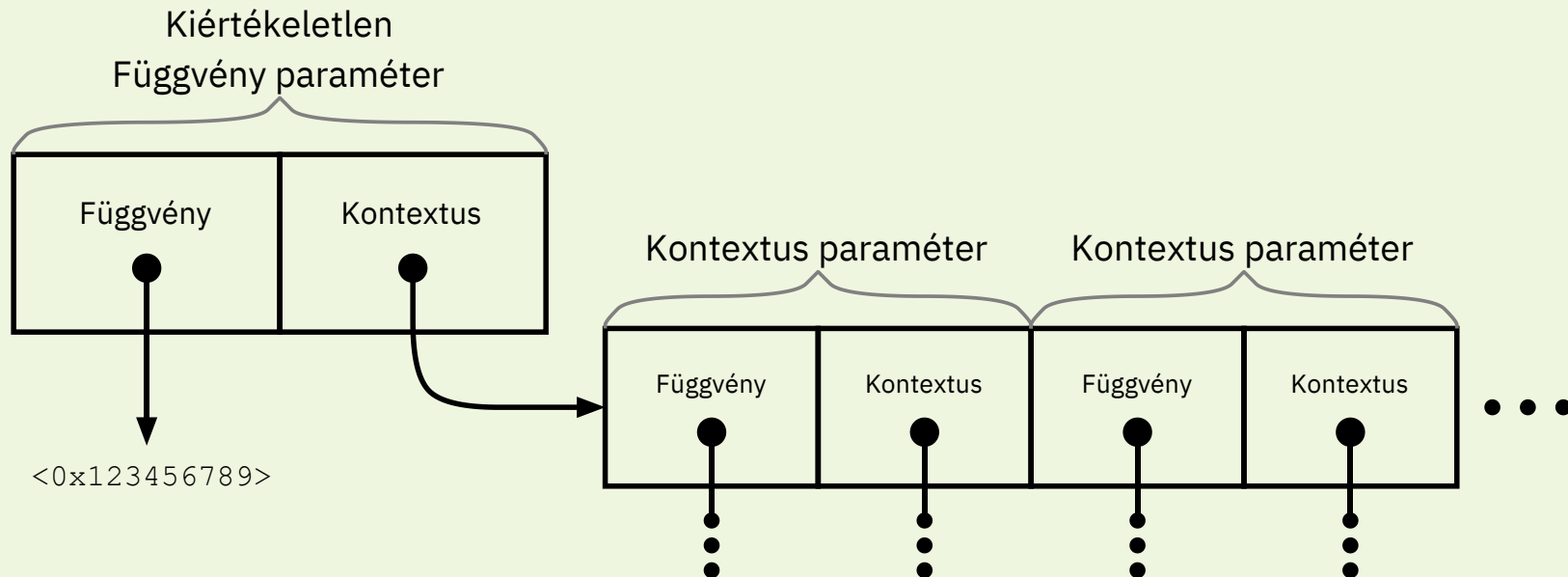
- “Platform független assembly”

- Mindent “csak” erre kell redukálni, és kész vagyunk

```
define i64 @_C2if3(ptr %cond,  
                  ptr %true,  
                  ptr %false) {  
eval:  
    %cond.real =  
        call i64 @_c4__arg_loader(  
            ptr %cond)  
    %isfalse = icmp eq  
                i64 %cond.real,  
                u0x7FF0000000000000  
    br i1 %isfalse,  
        label %branch_f,  
        label %branch_t
```

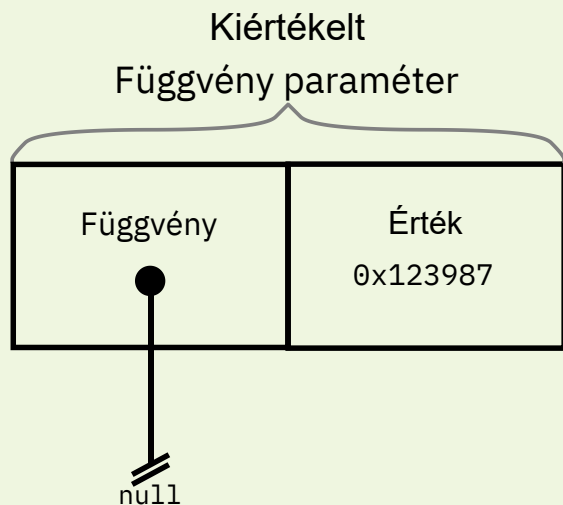
LLVM IR: Lustaság

- C4 lusta (lazy)
- LLVM IR szorgos (eager)



LLVM IR: Lustaság

- C4 lusta (lazy)
- LLVM IR szorgos (eager)



LLVM IR: Closure

- Függvények hivatkozhatnak olyan értékekre, amelyek előttük kerültek definiálásra
 - Closure
- Híváskor kellene
- Kontextus paraméter
 - Tömb “paraméterekből”
 - Egyes mezők azonosan működnek, mint a lusta paraméterek (ld. előbb)
 - Hivatkozottsági sorrendben

```
let x prompt ">"
let y x + 40
println "start"
let f/1 { |x|
  println y
  x
}
println f "kint"
```

```
start
> 2
42
kint
```




Kérdések?

- C4 nyelv
 - LISP-ek
 - Minden függvény
 - Szemantika
- LLVM & LLVM IR
 - Előnyök
 - Lustaság
 - Closure